

Java Interface Lab Assignment Answers

Problem 1

Part A

1. Should Animal be an abstract class or an interface? Why?

Answer:

Animal should be an abstract class because all animals share common data (name) and common methods (eat() and sleep()). The makeSound() method is different for each animal, so it is declared as abstract.

2. Should Swimmer and Flyer be abstract classes or interfaces? Why?

Answer:

Swimmer and Flyer should be interfaces because they represent optional abilities. Not every animal can swim or fly, and interfaces allow multiple capabilities to be added to a class.

Part C

1. Why can't Dog extend Animal and Swimmer?

Answer:

Java allows a class to extend only one class. However, it can implement multiple interfaces, so Dog extends Animal and implements Swimmer.

2. Why does sleep() belong in Animal instead of being copied into every class?

Answer:

The sleep() method has the same implementation for every animal. Keeping it in the abstract class avoids duplicate code and improves reusability.

3. Why is makeSound() abstract?

Answer:

Different animals produce different sounds. Therefore, there is no single implementation that can be shared by all animals.

Problem 2

Part A

1. Why is Payment an abstract class?

Answer:

Payment is an abstract class because all payment methods share common data and behavior, such as transaction ID generation, account validation, and authorization. At the same time, each payment method processes payments differently.

2. Why are Refundable, RecurringBillable, and LoyaltyPointsEarner interfaces?

Answer:

These are interfaces because they represent optional capabilities. Not every payment method supports refunds, recurring billing, or loyalty points, so interfaces provide flexibility.

Part C

1. Why can't validateAccountDetails() and the transaction ID generator be in an interface?

Answer:

They contain shared implementation that is common to all payment methods. An abstract class is the appropriate place to keep shared state and common logic.

2. Why shouldn't Refundable and RecurringBillable be abstract classes?

Answer:

If they were abstract classes, a payment class could not inherit from both them and Payment because Java supports only single inheritance. Interfaces solve this limitation.

3. Where should the common logging feature be added?

Answer:

It should be added to the Payment abstract class because the logging logic is identical for all payment methods and can be shared.

4. Why can CryptoPayment extend Payment and also be a LoyaltyPointsEarner?

Answer:

Java allows a class to extend one abstract class and implement multiple interfaces. If LoyaltyPointsEarner were an abstract class, CryptoPayment could not extend both classes.